Automatic Adjusting Global Similarity Measures in Learning CBR Systems

Stuart G. Ottersen and Kerstin Bach

Department of Computer Science
Norwegian University of Science and Technology (NTNU),
Trondheim, Norway
stuart.gallina.ottersen@ntnu.no,
http://www.ntnu.no/idi

Abstract. This paper explores how learning case-based reasoning (CBR) systems are affected by updating similarity measures. We create CBR systems using the local-global principle and we investigate (1) how adding new cases changes the CBR system's performance and (2) how this drift can be mitigated through updating the similarity measure, especially adapting feature weights for weighted sums. We aim to provide transparent measures to show when the knowledge containers drift apart to indicate when an update is necessary. We, therefore, explore the effect feature weight has on predictive performance and the knowledge containers in online learning CBR systems. Following this, we present a method to minimize updating feature weights while the case base grows while maintaining performance. The performance is compared to two baselines: never updating and always updating. Our experiments with public datasets show that a smart updating strategy catches the drifting of case base content and similarity measures well.

Keywords: XCBR, Similarity Learning, Global Weights, Explainable AI

1 Introduction

Case-based reasoning (CBR) uses previous experience and its adaptations to solve new, similar problems [1]. The previous experience is captured in cases containing the problem description and its solution. The solutions are retrieved by comparing the new and old problems using similarity measures. CBR systems are frequently used in decision support for more complex scenarios. This often warrants using domain experts when developing the knowledge representations used in the systems, such as similarity measures, case representations, or adaptation knowledge. Recently, data-driven methods have been introduced to create similarity measures from existing datasets [21,17,11]. Traditionally, CBR systems are built initially, and only a few have active learning phases. In scenarios where the system should learn, it is crucial to ensure that the knowledge representations represent the content of the case base well. If learned cases introduce

novel information, the system's vocabulary, similarity measures, and adaptation knowledge might need to be updated. This paper addresses the identification of drifts between a growing case base and similarity measures focusing on data-driven CBR systems. We explore data-driven approaches to detect change and then update existing similarity measures to avoid the high cost of having domain experts update the similarity measures manually. Similarity measures are often limited to the scope of the data present when they are built, possibly causing a more narrow domain than an expert would have modeled. The creation of data-driven similarity measures relies on: *Problem-solution regularity* and the *representativeness assumption*, which can be informally stated as "Similar problems have similar solutions" and "The case base is a representative sample of the target problem space" respectively [14]. A CBR system's case base can grow dynamically while learning, so the size and content can shift considerably, causing these core assumptions to be broken. Concept-drift causes the breach of said assumptions [22]. This can happen in multiple ways as described in [14] where our work will focus on how the distribution of the problems can cause concept-drift. This drift can then be reflected in a degradation in performance as the similarity measures no longer cover the problem space and/or have a poor representation of its distribution.

This paper explores the effect of updating similarity measures when a system learns. The tested system is built on the local-global principle [5], focusing mainly on updating the global similarity measure. Our main focus here lies on adapting the feature weights for weighted sums. Secondly, we aim to provide transparent measures that show when the knowledge containers drift apart to indicate that an update is necessary.

Therefore, we explore the effect of feature weights on predictive performance and the knowledge containers in online learning CBR systems. We then present a method to minimize updating these feature weights as the case base grows while maintaining performance. The performance is then compared to two baselines. This paper is structured as follows: in section 2 we discuss relevant work, followed by section 3 where the CBR system implementation is explained in detail. We then elaborate on the experimental setup and results in section 4 before discussing these results in section 5 and then concluding in section 6.

2 Related Work

Aamodt and Plaza [1] describe the four phases of case-based reasoning: First, a set of most similar cases are *retrieved*, then they can be *reused* to suit the problem description better. In the learning phase of a CBR system, cases are *revised* before they are *retained* and hence made available for future retrievals. The retrieval phase of CBR systems relies highly on similarity measures that work well, meaning they must represent the information in the case base well to reach the most relevant cases as described by Smyth and Keane [19]. This becomes a challenge as the case base grows due to concept drift or, in other words, a change in the data distribution. As presented by Lu et al. in [15], there

are three main types of approaches when detecting drift. These are error rate-based, data distribution-based, and multiple hypothesis test-based. Our work is error rate-based, where Gata et al.'s [12] is probably conceptually the most similar as it uses a warning system to detect a distribution change window on which the induction of a new model is performed. This window is set dynamically to capture as much of the "new distribution" as possible. Bifet and Gavaldà [6] again work on dynamic windows, which are now based on the stationarity of the data. Rather than training new models, it uses incremental models that update as the detection triggers it. More closely related to CBR, we have Leake and Schack [14] that focuses on *predictive case discovery*, meaning predicting what cases will be important for future problems as the distribution drifts. It functions more as a preemptive tool that tries to anticipate the drift rather than being reactive. Other relevant work in CBR includes applying CBR systems themselves for concept drift tracking such as Cunningham et al. [7] and Delany et al. [8]. The former of these uses the user to label spam and then periodically retrains to avoid concept shift, while the latter reselect features periodically and dynamically updates the case base tp reduce the error rate. As for the implementation of the system itself, there are similarities to Bayrak and Bach[4] where a black-box model is used as a twin to set global similarity measure weights by using SHAP-values. There is also considerable work done on concept drift outside of CBR, especially in streaming data in Lu et al.[15], but also more generally in statistics in Pollak [18].

3 Rolling Window Updates to Global Similarity (RUGS)

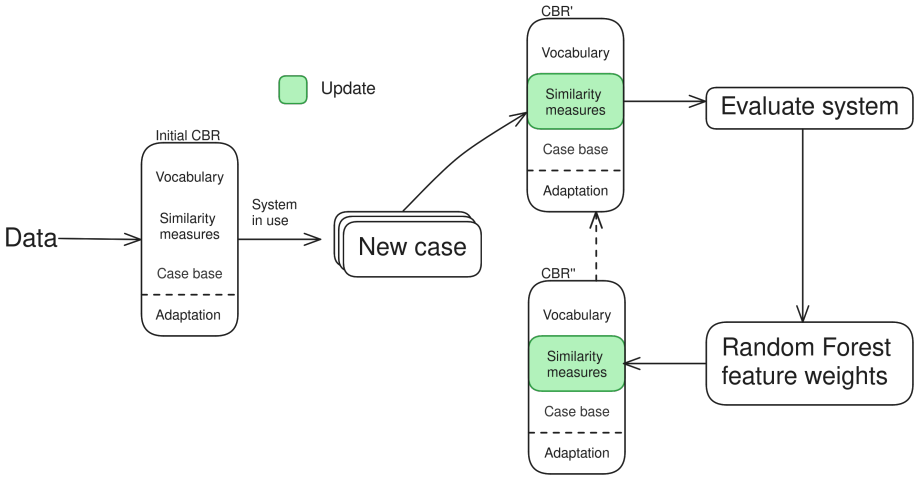


Fig. 1: Process of how an initial CBR system is created, cases are learned over time, and new global weights are determined.

The process used to build CBR systems in our scenario can be seen in Figure 1. An initial set of cases is used to construct the vocabulary and to create the similarity measures, this set is subsequently inserted into the case base. When this system is in use and new cases are introduced, the system will be updated and evaluated. The update consists of directly setting the feature attributions from a random forest classifier as the feature weights of the CBR system's global similarity measure. This process is then repeated each time new cases are added to the case base, giving us a CBR system that is iteratively becoming larger and with a similarity measure that is continuously updated to best represent the population in the case base.

3.1 Vocabulary

The vocabulary for numeric features is defined by calculating the minimum and maximum value for each feature over the entire dataset. For symbolic features, the full list of possible values is required to define the value range in the case representation. In this work, we will focus on the global similarity measures, so updating the value ranges is not considered part of the updating strategy. Setting the value ranges is considered trivial and can often be done automatically or by human experts up-front.

3.2 CBR System Learning

In this work, we explore how the case base grows during the retention phase as cases are added, (1) how this affects the performance, (2) in what way we can detect and mitigate performance loss, and (3) how to adjust the global similarity weights. The case base is initially empty or has a relatively small number of initial cases and will then grow as new cases are introduced. As new cases are added, the information contained in the case base will grow and might shift with time as the data distribution changes. As the CBR system is learning, this shift can cause a misalignment between the data present in the case base and the feature weights. This can cause a change in predictive behavior and possibly a decrease in performance. Updating the global similarity measures as new data is added is a good strategy but can be resource-demanding both computationally and cause delays in scenarios where models need to be approved before being set into production. Finding a balance between maintaining performance and updating the weights is, therefore, important. This can be seen in Figure 2, where one can see that continuously updating the feature weights will lead to better performance at a high cost, while not updating at all will allow the global similarity measures to become outdated and poorly represent the data. A system that updates only when necessary can maintain performance while reducing the number of updates, as seen in the green line in the figure.

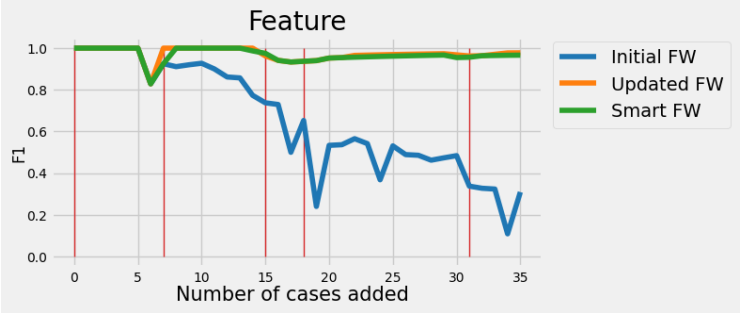


Fig. 2: Showing the difference between naive approaches and intelligent feature weight updating. The red line represents when the global similarity weights are updated. This is just an example, so the axes units are arbitrary, and the figure is meant only to show the concept of a smart updater.

3.3 Defining local similarity measure

Local similarity measures are frequently created by domain experts in the application domain of the CBR system being built, but this can be expensive, so there are a series of data-driven approaches that do this. The one applied in the presented system uses polynomial functions, distance functions, and the interquartile range to set the steepness of the similarity decline in said function. This is seen in 3 where the $IQR = 4$ is aligned to that the polynomial function approximately returns 0.3 and the $range = 8$ such that it approximately returns 0. In other words, the interquartile range is used to set the degree of the polynomial function to ensure that the entire similarity range $[0, 1]$ is used[21,16].

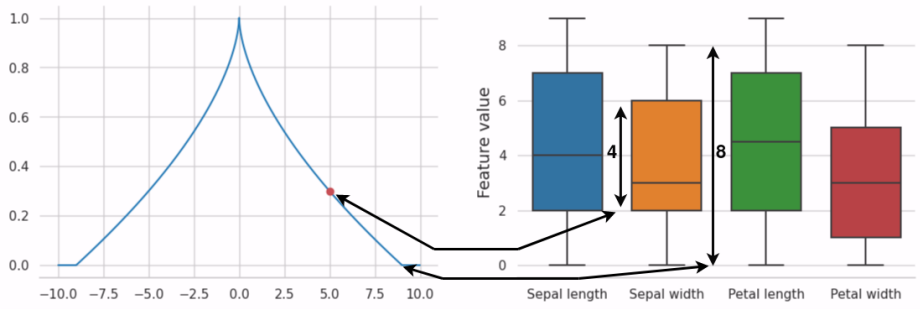


Fig. 3: Example showing how IQR is used to set polynomial function.

3.4 Defining global similarity measure

The global similarity measure can be seen as a weighted average of the similarity values received from the local similarity measures. In the presented work, these are calculated using the feature attributions of a random forest classifier on the data present in the case base. The feature attribution from the random forest classifier is directly used to set the global similarity functions. The feature attributions returned by the random forest represent how important each feature is in separating the classes present in the dataset, using these as the feature weights should work well in a CBR system. This method of updating the similarity measure is seen in Figure 1 where the steps in the system are:

1. New cases are created while the system is online
2. Retention: cases are added to case base
3. System performance is evaluated
4. Feature attributions are obtained from random forest classifier
5. Feature weights are updated into global similarity measures

3.5 The RUGS approach

The main contribution of this work is the RUGS method for updating the feature weights using a rolling window of the performance of the CBR system. The aim of RUGS is to avoid unnecessary updates to the CBR system, as each update can in practice, lead to extensive test and validation work. With RUGS we are identifying when the drift between the case base and similarity measures is large enough to suggest an update.

The predictive performance of a CBR system is measured applying a leave-one-out cross-validation measuring the F1-micro score. A rolling window calculates the average score over the last five batches added to the case base as seen in Algorithm 1. If the performance over the last five rolling windows has decreased monotonically and the difference between the first and last of these five values is larger than the standard deviation of the same five values, then a feature weight update is triggered. To discourage frequent feature weight updates, the system does not update if there is no improvement larger than the standard deviation from the last update to the most recent value. In other words, if the last update was quite recent and it did not have an effect, then updating now probably will not either. The *LastUpdate* in Algorithm 1 is a function that fetches the performance of the last batch in the window that updated the feature weights. If there is no update in the current window, then the entire if-statement is ignored (this is not shown in the pseudocode to maintain readability).

Algorithm 1 RUGS

```
function UpdateFeatureWeights
    rollingwindow ← F1 scores for 5 past values
    if LastUpdate(rollingwindow) + Std(rollingwindow) < rollingwindow[−1]
        return false
    if MonotonicallyDecreasing(rollingwindow) and
        rollingwindow[0] > rollingwindow[−1] + Std(rollingwindow)
        return true
    return false
```

4 Experiments and results

4.1 Experimental setup

The experiment aims to measure the change in predictive behavior as cases are introduced to the system with how frequently feature weights are updated. This is done through batching. Cases are introduced to the case base in batches of size 3% of the dataset with a minimum threshold of 5. After each batch, a leave-one-out cross-validation on the data in the case base is carried out, and the F1-micro and accuracy scores are calculated. Following the addition of the first batch, the global similarity is set by directly using the feature attributions from a random forest classifier. Consequently, after adding each batch, the system chooses whether to update the global similarity measures. The weights defining the global similarity measures are updated using feature attribution obtained from a random forest classifier trained on cases already added to the case base. Three strategies are tested: no update after the first batch, updates after every batch, or using *RUGS*.

In real-world applications, various case learning strategies can be implemented: either all cases are added, only cases with a certain difference, or manually reviewed cases. In this experiment, we want to create different batches to show how the RUGS method picks up a change. Therefore we simulate the concept drift so that learned cases vary from those existing in the case base. In our simulation, we imitate case learning by creating batches as follows: a single feature is selected and sorted in ascending order on its feature values. This simulates a drift in the data introduced to the case base. The batch size is set to 3% of the original dataset with a minimum threshold of five, which are then added to the case base. This is repeated in separate experiments for each feature individually, as represented in Figure 4. Sorting on a single feature at a time creates an artificial and unnatural distribution for the data to be introduced, but is performed to showcase an extreme and how this can affect performance. Performance is captured using the F1-score, this allows us to indirectly measure the skew in the solution space relative to the problem space that is actually being shifted artificially.

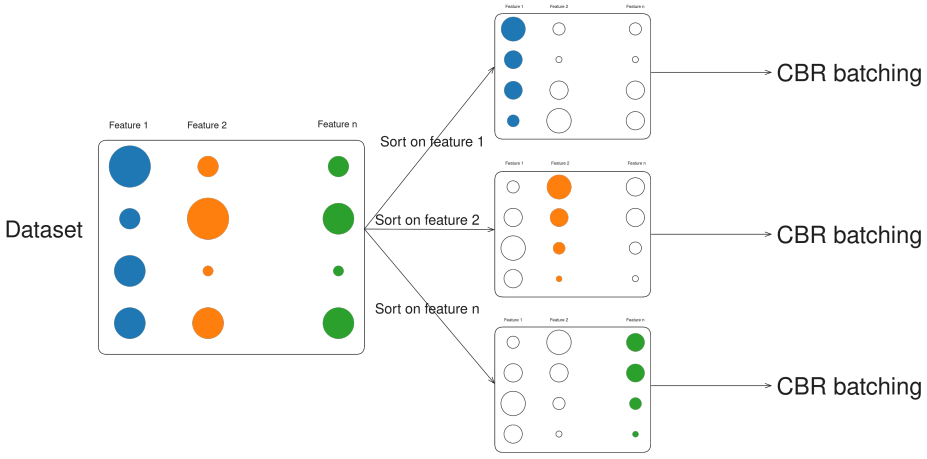


Fig. 4: Batch creation for the experiments: First, data is sorted by feature value, then batches are created that are then incrementally added to the CBR system.

4.2 Data

The experiment is performed on seven datasets which are presented in Table 1. The selected datasets offer a good baseline to show that the effect generalizes over multiple scenarios. The chosen open datasets allow for reproducibility and understandability while capturing a breadth of domains and data types in the features.

Name	# classes	# cases	# of features (cat./bin./num.)	Class ratio
Banking[13]	2	1000	0:0:24	1:0.42
Glioma[20]	2	839	22:0:1	1:0.72
Iris[9]	3	150	0:0:4	1:1:1
Tictactoe[3]	2	958	9:0:0	1:0.53
Wdbc[23]	2	569	1:0:29	1:0.59
Wine[2]	3	178	0:0:13	1:0.83:0.67
Zoo[10]	7	101	15:1:0	1:0.48:0.31:0.24:0.20:0.12:0.10

Table 1: Overview of the open datasets used in the experiment

A CBR system is built for each dataset by setting the vocabulary given the entire dataset and defining the local similarity measures using the approach presented in [21,16]. Following this, the batching is performed as described in Section 4.1, and the global similarity is updated by directly using the random forest feature attributions as feature weights.

4.3 Results

The effectiveness of *RUGS* is evident in the results of the experiment shown in Table 2. Here we see that the F1-score remains competitive with the continuously updated feature weights while using considerably fewer updates. The performance of the CBR systems created in this experiment is comparable to the performance ranges given by UCI for the datasets where this information is given.

Strategy	Initial		Updated		RUGS	
Dataset	F1-score	Number of updates	F1-score	Average number of updates	F1-score	Number of updates
Banking	0.69±0.05	1	0.71±0.05	34	0.7±0.05	2.08
Glioma	0.87±0.06	1	0.88±0.05	34	0.88±0.05	5.04
Iris	0.7±0.25	1	0.95±0.07	30	0.94±0.08	3.75
Tictactoe	0.7±0.17	1	0.81±0.07	34	0.78±0.11	4.11
Wdbc	0.87±0.14	1	0.96±0.02	34	0.96±0.04	3.63
Wine	0.7±0.27	1	0.97±0.03	36	0.95±0.08	3.46
Zoo	0.73±0.28	1	0.91±0.09	21	0.84±0.2	1.88

Table 2: Showing results calculated on all features and all batches. Zoo has fewer updates due to the minimum size of 5 for the batch size.

A more in-depth analysis is performed using a boxplot, Figure 5, where we see that, in general, RUGS can perform similarly to always updating the feature weights. The variance is also very comparable, while for the initial feature weight update, the performance tends to be considerably worse and has a higher variance. For some of the datasets, one can also see that the three methods have similar performance overall, probably caused by very equal distributions of the data, so there is very little drift when adding the batches.

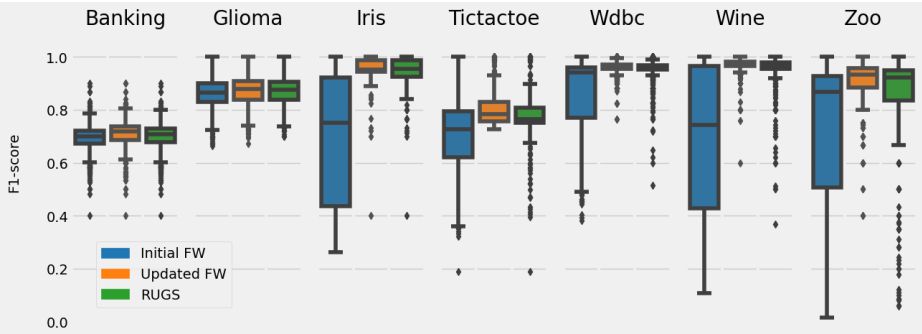


Fig. 5: Showing boxplot of results for each dataset and each feature weight updating strategy. Calculated over all features and batches.

In Figure 6 we take a closer look at the iris dataset. Each plot represents one experiment where the data was introduced in the order of the named feature. The F1-score is then plotted over the batch number, and one can notice that the RUGS efficiently detects a drop in performance and ensures that the performance of the model rebounds.

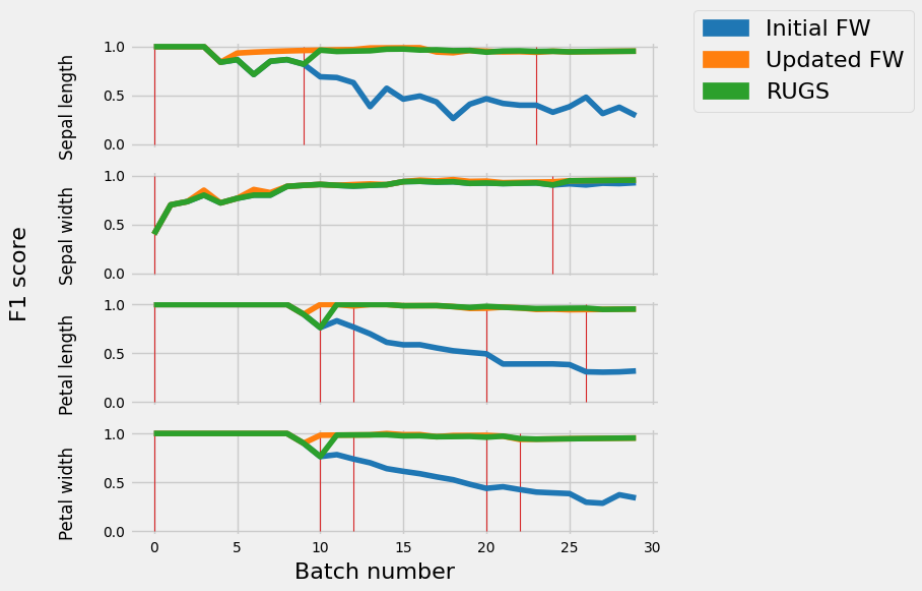
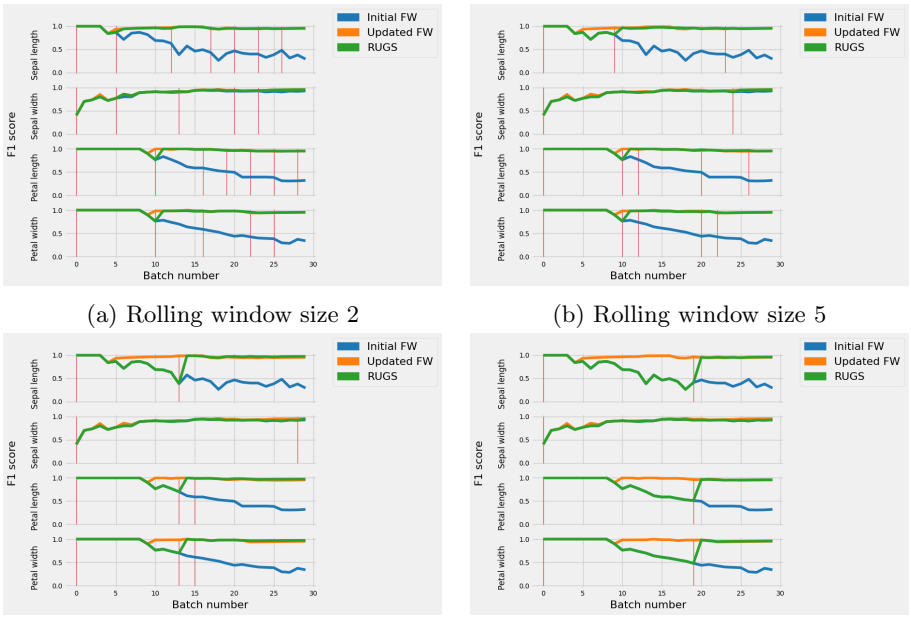


Fig. 6: Showing change in F1 score for each feature over batch number, the red lines represent feature weight updates performed .

The main parameter of the model is the window size, which functions as a weighting parameter between feature weight updates and F1-score. This also means that the performance of the model is reliant on selecting a fitting rolling window size, as can be seen in Figure 7, where it is evident that selecting an excessively small window will cause an exaggerated number of updates, while a large window will allow for periods of poorer predictive performance. The batch size is somewhat arbitrarily set to 3% of the original dataset, which is not possible in a real-world scenario where the actual dataset size is unknown. This is a parameter that could affect the performance as well. Still, the focus in the experiments is maintained on window size as the batch parameter would, in a real-world scenario, be highly variable.



(a) Rolling window size 2

(b) Rolling window size 5

(c) Rolling window size 7

(d) Rolling window size 10

Fig. 7: Showing the effect of rolling window size on predictive performance for iris dataset.

The trends seen in Figure 6 and Figure 7 generalize over the other datasets, where smaller windows will prevent longer drops in performance.

Figure 8 and Figure 9 show the results for the Wine and Tic Tac Toe datasets, which are two examples that show that some features do not require feature updates and will maintain performance. This is especially evident in the Tic Tac Toe dataset where multiple plots have all three lines overlapping. While for the Wine dataset the update of the feature weights has less impact. This indicates that the window size can be varied considerably depending on the scenario. Looking at the overall feature importance at the end of the experiments we see that the features with a low feature importance are the ones that tend to have little effect from updating the feature importance. An example of this is the feature *Mrs* in Figure 9.

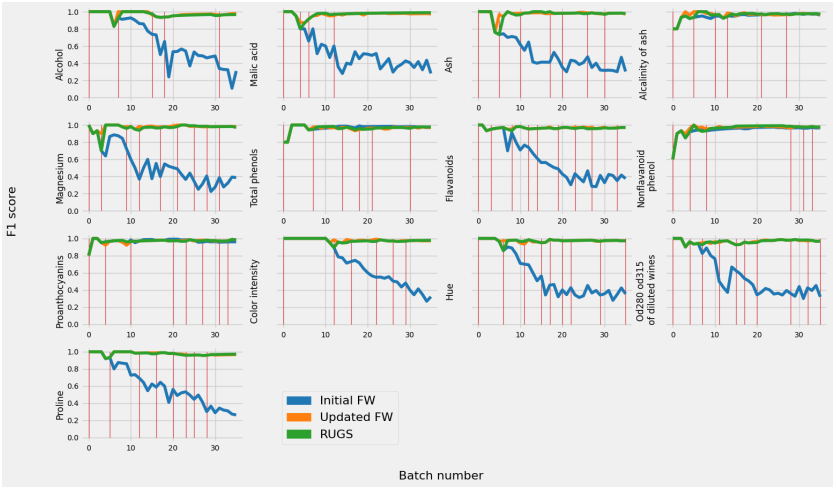


Fig. 8: Rolling window size 2 for Wine dataset

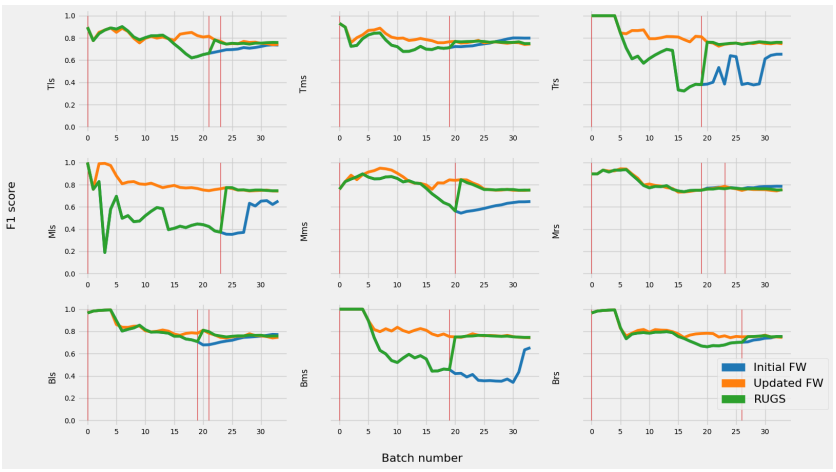


Fig. 9: Rolling window size 10 for Tic Tac Toe dataset

To further emphasize the value RUGS brings we compare the computational cost of updating the feature weights compared to the cost of running RUGS. One can see in Table 3 that RUGS takes about one tenth the time to run, making the trade off worth it in all of the tested scenarios.

Dataset	Feature weight calculation	RUGS	Calculation/RUGS
Banking	0.35±0.0	0.04±0.01	0.13±0.02
Glioma	0.36±0.0	0.04±0.01	0.11±0.01
Iris	0.34±0.01	0.02±0.0	0.05±0.01
Tictactoe	0.36±0.0	0.03±0.0	0.09±0.01
Wdbc	0.32±0.01	0.05±0.01	0.14±0.02
Wine	0.34±0.01	0.03±0.0	0.08±0.01
Zoo	0.35±0.0	0.02±0.0	0.05±0.01
Mean	0.34±0.0	0.03±0.01	0.09±0.01

Table 3: Showing the running time in seconds for the time random forest uses to calculate the feature attributions and average running time for RUGS

5 Discussion

The results show that *RUGS* can identify drops in performance and improve the predictive performance by updating the global similarity measure. *RUGS* relies on selecting an adequate window size for the scenario, which simultaneously allows the system to be adaptable for various scenarios depending on the update cost and how critical slight performance loss is to the system.

Sorting features with an overall low feature importance, meaning feature weights calculated over the entire dataset, seems to have little effect on performance. The difference in performance between the system with updated and old feature weights is quite negligible. This is, for example, seen with the feature "sepal width" in the iris dataset in Figure 6. This trend is noticed in most of the datasets. A converse effect is seen in features with high feature importance, where they separate the classes so well that the initial feature attributions are all 1, as only one class is represented in the initial batch. This is slightly unrealistic as training a prediction system with only one class is pointless but may give some information about the behavior of systems with highly imbalanced datasets. Some datasets with many features seem to have less performance increase from frequent updating, as noticed in Table 2. This might be caused by how the random forest classifier distributes weight early in the batching process, as randomly spreading the weight is a functioning strategy. This works as the feature importance seems to be more evenly spread amongst the features(this is at least the case with the banking and glioma datasets) than with the other datasets, where some features are considerably more important than others. The experimental setup might also create some bias here as single features that separate the classes well will cause a larger performance variance as the data is introduced and sorted on a single feature.

The order in which cases are introduced is somewhat artificial and does not necessarily represent a realistic scenario. Other scenarios were also considered, such as clustering and assuming a cluster represents a case to then introduce outliers first or introducing points close to the cluster centers first. Random batching

was also attempted, but due to the simplicity of the data, the system was very quickly able to represent the entire dataspace in its case base, making any feature weight updates unnecessary. The chosen scenario is somewhat extreme, but the others are equally arbitrary, as it is hard to know in what order the data will be introduced. It was also chosen to test on open and known datasets and stay general without focusing on specific scenarios. At the same time, the simplicity and clean nature of the data does potentially introduce some effect on the result, potentially offering more stable results than more complex real-world datasets. The average performance over all batches, as well as the standard deviation for the three updating strategies, can be seen in Table 2.

The datasets used are all classification problems, as adapting the system for regression would require the use of a random forest regressor as well as a different performance measure. The system itself is adaptable as these should be relatively simple changes to make but have not been included for understandability's sake.

Looking back at the research questions, these can briefly be summarized as follows: **(1) How adding cases affects the performance:** The performance change varies depending on the order in which data is introduced. If the distribution changes significantly and the feature in which the distribution change is done is important to separate the classes, this will significantly affect the performance. **(2) in what way can we detect and mitigate performance loss:** By using rolling windows and detecting a decrease in performance over multiple batches, we can effectively detect when to update the weights while maintaining performance. **(3) how to adjust the global similarity weights:** Using the feature attributions of a random forest classifier and updating these when an update is deemed necessary seems to be an effective way of maintaining aligned knowledge containers.

6 Conclusion

In this paper, we present a novel method to detect and update feature weights in learning online CBR systems, allowing for less frequent feature weight updates while maintaining performance. The approach is compared to two baselines on seven datasets and can score comparably to the method updating feature weights after every case addition.

.Future work should explore more specific scenarios to see if the method is applicable in a more complex context. Exploring the effect the window size has on performance in more detail could also be interesting, as this parameter could function as a proxy for the sensitivity of the feature weight updates. Comparing the work to more complex baselines would also be a valuable addition.

References

1. Aamodt, A., Plaza, E.: Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches. AI Communications 7(1), 39–59 (1994), https://www.medra.org/servlet/aliasResolver?alias=iospress&doi=10.3233/AIC-1994-7104
2. Aeberhard, S., Forina, M.: Wine. UCI Machine Learning Repository (1991), DOI: https://doi.org/10.24432/C5PC7J
3. Aha, D.: Tic-Tac-Toe Endgame. UCI Machine Learning Repository (1991), DOI: https://doi.org/10.24432/C5688J
4. Bayrak, B., Bach, K.: A Twin XCBR System Using Supportive and Contrastive Explanations. In: Workshop on Case-Based Reasoning for the Explanation of Intelligent Systems at ICCBR2023 (2023)
5. Bergmann, R.: Experience management: foundations, development methodology, and Internet-based applications. No. 2432 in Lecture notes in computer science, Lecture notes in artificial intelligence, Springer, Berlin ; New York (2002)
6. Bifet, A., Gavaldà, R.: Learning from Time-Changing Data with Adaptive Windowing. In: Proceedings of the 2007 SIAM International Conference on Data Mining. pp. 443–448. Society for Industrial and Applied Mathematics (Apr 2007), https://epubs.siam.org/doi/10.1137/1.9781611972771.42
7. Cunningham, P., Nowlan, N., Delany, S.J., Haahr, M.: A Case-Based Approach to Spam Filtering that Can Track Concept Drift
8. Delany, S.J., Cunningham, P., Tsymbal, A., Coyle, L.: A case-based technique for tracking concept drift in spam filtering. Knowledge-Based Systems 18(4-5), 187–195 (Aug 2005), https://linkinghub.elsevier.com/retrieve/pii/S0950705105000316
9. Fisher, R.A.: Iris. UCI Machine Learning Repository (1988), DOI: https://doi.org/10.24432/C56C76
10. Forsyth, R.: Zoo. UCI Machine Learning Repository (1990), DOI: https://doi.org/10.24432/C5R59V
11. Gabel, T., Godehardt, E.: Top-Down Induction of Similarity Measures Using Similarity Clouds. In: Hüllermeier, E., Minor, M. (eds.) Case-Based Reasoning Research and Development, vol. 9343, pp. 149–164. Springer International Publishing, Cham (2015), http://link.springer.com/10.1007/978-3-319-24586-7_11, series Title: Lecture Notes in Computer Science
12. Gama, J., Medas, P., Castillo, G., Rodrigues, P.: Learning with Drift Detection. In: Hutchison, D., Kanade, T., Kittler, J., Kleinberg, J.M., Mattern, F., Mitchell, J.C., Naor, M., Nierstrasz, O., Pandu Rangan, C., Steffen, B., Sudan, M., Terzopoulos, D., Tygar, D., Vardi, M.Y., Weikum, G., Bazzan, A.L.C., Labidi, S. (eds.) Advances in Artificial Intelligence – SBIA 2004, vol. 3171, pp. 286–295. Springer Berlin Heidelberg, Berlin, Heidelberg (2004), http://link.springer.com/10.1007/978-3-540-28645-5_29, series Title: Lecture Notes in Computer Science
13. Hofmann, H.: Statlog (German Credit Data). UCI Machine Learning Repository (1994), DOI: https://doi.org/10.24432/C5NC77
14. Leake, D., Schack, B.: Towards Addressing Problem-Distribution Drift with Case Discovery. In: Massie, S., Chakraborti, S. (eds.) Case-Based Reasoning Research and Development, vol. 14141, pp. 244–259. Springer Nature Switzerland, Cham (2023), https://link.springer.com/10.1007/978-3-031-40177-0_16, series Title: Lecture Notes in Computer Science
15. Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G.: Learning under Concept Drift: A Review. IEEE Transactions on Knowledge and Data Engineering pp. 1–1 (2018), https://ieeexplore.ieee.org/document/8496795/

16. Marín-Veites, P., Bach, K.: Explaining CBR Systems Through Retrieval and Similarity Measure Visualizations: A Case Study. In: Keane, M.T., Wiratunga, N. (eds.) Case-Based Reasoning Research and Development, vol. 13405, pp. 111–124. Springer International Publishing, Cham (2022), https://link.springer.com/10.1007/978-3-031-14923-8_8, series Title: Lecture Notes in Computer Science
17. Mathisen, B.M., Aamodt, A., Bach, K., Langseth, H.: Learning similarity measures from data. Progress in Artificial Intelligence 9(2), 129–143 (Jun 2020), http://arxiv.org/abs/2001.05312, arXiv:2001.05312 [cs, stat]
18. Pollak, M.: Optimal Detection of a Change in Distribution. The Annals of Statistics 13(1) (Mar 1985), https://projecteuclid.org/journals/annals-of-statistics/volume-13/issue-1/Optimal-Detection-of-a-Change-in-Distribution/10.1214/aos/1176346587.full
19. Smyth, B., Keane, M.T.: Remembering to forget: a competence-preserving case deletion policy for case-based reasoning systems. In: Proceedings of the 14th international joint conference on Artificial intelligence - Volume 1. pp. 377–382. IJCAI'95, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1995), event-place: Montreal, Quebec, Canada
20. Tasci, E., Camphausen, K., Krauze, A.V., Zhuge, Y.: Glioma Grading Clinical and Mutation Features. UCI Machine Learning Repository (2022), DOI: https://doi.org/10.24432/C5R62J
21. Verma, D., Bach, K., Mork, P.J.: Similarity Measure Development for Case-Based Reasoning–A Data-Driven Approach. In: Bach, K., Ruocco, M. (eds.) Nordic Artificial Intelligence Research and Development, vol. 1056, pp. 143–148. Springer International Publishing, Cham (2019), http://link.springer.com/10.1007/978-3-030-35664-4_14, series Title: Communications in Computer and Information Science
22. Widmer, G., Kubat, M.: Learning in the presence of concept drift and hidden contexts. Machine Learning 23(1), 69–101 (Apr 1996), http://link.springer.com/10.1007/BF00116900
23. Wolberg, W., Mangasarian, O., Street, N., Street, W.: Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository (1995), DOI: https://doi.org/10.24432/C5DW2B